

Docker Container Environment

Implementation Guide

Subject: Web Application Development (WAD)

Practical: 2B

This document provides step-by-step instructions to set up and work with Docker container environments, including Docker Desktop, basic commands, Dockerfile creation, multi-container orchestration with Docker Compose, and optional NVIDIA Container Toolkit for GPU-accelerated workloads.

Generated for academic practical implementation

WAD Practical 2B: Docker Container Environment

Table of Contents

1. Prerequisites and System Requirements
2. Installing Docker Desktop on Windows
3. Enabling WSL 2 Backend (Required for GPU support)
4. Basic Docker Commands
5. Creating Your First Dockerfile
6. Building and Running Containers
7. Docker Compose: Multi-Container Setup
8. NVIDIA Container Toolkit (Optional - GPU)
9. Hands-On Exercises
10. Common Troubleshooting

1. Prerequisites and System Requirements

1.1 System Requirements for Docker Desktop

Before installing Docker, ensure your Windows machine meets the following requirements:

Operating System: Windows 10/11 (64-bit), version 19041 or higher.

Processor: 64-bit processor with Second Level Address Translation (SLAT).

RAM: Minimum 4 GB (8 GB or more recommended).

Virtualization: Hyper-V and Windows Hypervisor Platform must be enabled.

WSL 2: Windows Subsystem for Linux version 2 (recommended for best performance).

Disk Space: At least 10 GB of free space.

GPU (Optional): NVIDIA GPU with compute capability 5.0+ for NVIDIA Docker support.

1.2 Required Software / Accounts

- Docker Hub account (free): <https://hub.docker.com/>
- Windows Terminal or PowerShell (admin rights required)
- Visual Studio Code (recommended) with Docker extension

NOTE: If you are using Windows Home edition, WSL 2 backend is mandatory as Hyper-V is not available.

2. Installing Docker Desktop on Windows

Step 1: Download Docker Desktop

1. Open your web browser and go to <https://www.docker.com/products/docker-desktop/>
2. Click on 'Download for Windows - AMD64' (or ARM64 if applicable).
3. Save the installer (.exe file) to your Downloads folder.

Step 2: Run the Installer

1. Double-click the Docker Desktop Installer.exe file.
2. When prompted, ensure 'Use WSL 2 instead of Hyper-V' is checked (recommended).
3. Click OK and wait for the installation to complete. This may take several minutes.
4. If prompted, restart your computer after the installation finishes.

Step 3: Verify Installation

After restarting, open PowerShell or Command Prompt and run:

```
docker --version
docker-compose --version
```

You should see output similar to:

```
Docker version 26.x.x, build xxxxxxxx
Docker Compose version v2.x.x
```

WARNING: Docker Desktop must be running (icon in system tray) for commands to work.

3. Enabling WSL 2 Backend (Required for GPU support)

Step 1: Install WSL 2

Open PowerShell as Administrator and run:

```
wsl --install
```

This installs Ubuntu as the default distribution along with WSL 2 kernel. Restart your PC when prompted.

Step 2: Set WSL 2 as Default

```
wsl --set-default-version 2
```

Step 3: Configure Docker Desktop

1. Open Docker Desktop.
2. Go to Settings (gear icon) > General.
3. Ensure 'Use the WSL 2 based engine' is checked.
4. Go to Resources > WSL Integration.
5. Enable integration with your default WSL distro (e.g., Ubuntu).
6. Click 'Apply & Restart'.

NOTE: WSL 2 is essential for running Linux containers and is the only way to use NVIDIA Container Toolkit on Windows.

4. Basic Docker Commands

Practice these essential commands in PowerShell / Terminal:

Check Docker version

```
docker --version
```

List running containers

```
docker ps
```

List all containers (including stopped)

```
docker ps -a
```

List Docker images

```
docker images
```

Pull an image from Docker Hub

```
docker pull hello-world
```

Run a container

```
docker run hello-world
```

Run container interactively

```
docker run -it ubuntu bash
```

Run container in background

```
docker run -d -p 8080:80 nginx
```

Stop a running container

```
docker stop <container_id>
```

Remove a container

```
docker rm <container_id>
```

Remove an image

```
docker rmi <image_id>
```

View container logs

```
docker logs <container_id>
```

Execute command inside running container

```
docker exec -it <container_id> bash
```

NOTE: Use the first 3-4 characters of the container ID instead of typing the full ID.

5. Creating Your First Dockerfile

A Dockerfile is a text file containing instructions to build a Docker image. Let's create a simple web server.

Step 1: Create Project Directory

```
mkdir my-docker-app
cd my-docker-app
```

Step 2: Create a simple HTML file

```
notepad index.html
```

Paste the following content and save:

```
<!DOCTYPE html>
<html>
<head><title>Docker App</title></head>
<body>
  <h1>Hello from Docker Container!</h1>
  <p>This page is served by Nginx inside a Docker container.</p>
</body>
</html>
```

Step 3: Create the Dockerfile

```
notepad Dockerfile
```

Paste the following content and save (no file extension):

```
# Use official Nginx image as base
FROM nginx:alpine

# Copy custom HTML to Nginx web root
COPY index.html /usr/share/nginx/html/

# Expose port 80
EXPOSE 80

# Start Nginx when container launches
CMD ["nginx", "-g", "daemon off;"]
```

NOTE: The Dockerfile must be named exactly 'Dockerfile' with no extension.

6. Building and Running Containers

Step 1: Build the Docker Image

```
docker build -t my-nginx-app .
```

Explanation:

- '-t my-nginx-app' tags the image with name 'my-nginx-app'
- '.' means the Dockerfile is in the current directory

Step 2: Run the Container

```
docker run -d -p 8080:80 --name web-container my-nginx-app
```

Explanation:

- '-d' runs in detached (background) mode
- '-p 8080:80' maps host port 8080 to container port 80
- '--name web-container' assigns a friendly name

Step 3: Test in Browser

Open your browser and navigate to: <http://localhost:8080>

You should see your 'Hello from Docker Container!' message.

Step 4: Stop and Clean Up

```
docker stop web-container  
docker rm web-container  
docker rmi my-nginx-app
```

7. Docker Compose: Multi-Container Setup

Docker Compose allows you to define and run multi-container applications using a YAML file.

Step 1: Create docker-compose.yml

```
notepad docker-compose.yml
```

Paste the following content:

```
version: '3.8'

services:
  web:
    image: nginx:alpine
    ports:
      - '8080:80'
    volumes:
      - ./index.html:/usr/share/nginx/html/index.html
    container_name: compose-web

  db:
    image: mysql:8.0
    environment:
      MYSQL_ROOT_PASSWORD: rootpass
      MYSQL_DATABASE: testdb
    ports:
      - '3306:3306'
    container_name: compose-db

volumes:
  db_data:
```

Step 2: Start Services

```
docker-compose up -d
```

Step 3: Verify Running Services

```
docker-compose ps
```

Step 4: Stop and Remove Services

```
docker-compose down
```

NOTE: Ensure docker-compose.yml and index.html are in the same directory before running docker-compose commands.

WAD Practical 2B: Docker Container Environment

8. NVIDIA Container Toolkit (Optional - GPU)

NVIDIA Container Toolkit (nvidia-docker) allows Docker containers to access GPU resources. This is useful for machine learning, AI, and compute-intensive tasks.

Step 1: Verify NVIDIA GPU

```
nvidia-smi
```

If this command fails, update your NVIDIA drivers from <https://www.nvidia.com/drivers>

Step 2: Install NVIDIA Container Toolkit on WSL 2

Open your WSL 2 Ubuntu terminal and run these commands:

```
# Add NVIDIA package repositories
distribution=$(. /etc/os-release;echo $ID$VERSION_ID)
curl -s -L https://nvidia.github.io/nvidia-docker/gpgkey | sudo apt-key add -
curl -s -L https://nvidia.github.io/nvidia-docker/$distribution/nvidia-docker.list | sudo tee
/etc/apt/sources.list.d/nvidia-docker.list
```

```
# Install nvidia-docker2 package
sudo apt-get update
sudo apt-get install -y nvidia-docker2
```

```
# Restart Docker daemon
sudo systemctl restart docker
```

Step 3: Test GPU Container

```
docker run --rm --gpus all nvidia/cuda:12.0-base nvidia-smi
```

If successful, you will see your GPU details inside the container.

WARNING: NVIDIA Container Toolkit on Windows requires WSL 2 backend. It does NOT work with Windows containers or Hyper-V backend.

9. Hands-On Exercises

Exercise 1: Basic Container Lifecycle

1. Pull the 'hello-world' image.
2. Run it and observe the output.
3. List all containers and note the exited container.
4. Remove the exited container.
5. Remove the 'hello-world' image.

Exercise 2: Custom Node.js Application

1. Create a directory 'node-app'.
2. Inside it, create 'app.js' that prints 'Hello from Node.js' to the console.
3. Create a Dockerfile using 'node:18-alpine' as base.
4. Build and run the image.
5. Verify the output using 'docker logs'.

Exercise 3: Persist Data with Volumes

1. Run an nginx container with a volume mounted to /usr/share/nginx/html.
2. Create a custom index.html in the mounted host directory.
3. Verify the changes reflect in the browser.
4. Delete and recreate the container.
5. Confirm that the custom page persists.

Exercise 4: Docker Compose Web + DB Stack

1. Create a docker-compose.yml with PHP-Apache and MySQL services.
2. Connect the PHP application to MySQL using the service name as hostname.
3. Verify both containers can communicate on the default Docker network.

10. Common Troubleshooting

Docker Desktop fails to start

- Ensure virtualization is enabled in BIOS.
- Enable Hyper-V and Windows Hypervisor Platform in Windows Features.
- Ensure WSL 2 is installed and set as default.

Port already in use

- Change the host port mapping (e.g., -p 8081:80 instead of -p 8080:80).
- Find and stop the process using the port: `netstat -ano | findstr :8080`

Permission denied on WSL

- Run commands with 'sudo' or add your user to the docker group:
`sudo usermod -aG docker $USER && newgrp docker`

Image pull fails / no internet in container

- Check Docker Desktop network settings.
- Restart Docker Desktop.
- Verify proxy settings if behind a corporate firewall.

NVIDIA GPU not visible in container

- Ensure WSL 2 is used (not Hyper-V).
- Install latest NVIDIA drivers supporting WSL.
- Verify nvidia-docker2 is installed inside WSL distro.

Container exits immediately

- Check container logs: `docker logs <container_id>`
- Ensure the foreground process is specified in Dockerfile CMD/ENTRYPOINT.
- Use 'docker run -it' for interactive debugging.

End of Document

Follow each step carefully and verify outputs at every stage.

Refer to official Docker documentation for advanced topics:

<https://docs.docker.com/>